

mongodb-exercises-questions

MongoDB - Hands-On Exercises

Q1. Create a database called `shopDB` and switch to it.

Q2. Insert a single product document into a `products` collection with the following fields:

- `name`: "Wireless Mouse"
- `brand`: "Logitech"
- `price`: 29.99
- `category`: "Electronics"
- `stock`: 150
- `rating`: 4.5

Q3. Insert the following three products in one command:

- `{ name: "Mechanical Keyboard", brand: "Corsair", price: 89.99, category: "Electronics", stock: 60, rating: 4.7 }`
- `{ name: "USB-C Hub", brand: "Anker", price: 35.00, category: "Electronics", stock: 200, rating: 4.3 }`
- `{ name: "Desk Lamp", brand: "BenQ", price: 109.99, category: "Office", stock: 45, rating: 4.6 }`

Q4. Retrieve all documents from the `products` collection.

Q5. Find only the product named `"Wireless Mouse"`.

Q6. Update the `stock` of `"USB-C Hub"` to `180`.

Q7. Add a new field `"onSale": true` to **all** products in the `"Electronics"` category.

Q8. Delete the document where the name is `"Desk Lamp"`.

Q9. Find all products with a `price` greater than `30`.

Q10. Find all products with a `rating` between `4.4` and `4.8` (inclusive).

Q11. Find all products that belong to either the `"Electronics"` or `"Office"` category.

Q12. Find all products where the field `"onSale"` exists.

Q13. Find all products with `price` less than `100` AND `stock` greater than `100`. Return only the `name`, `price`, and `stock` fields (exclude `_id`).

Q14. Retrieve all products sorted by `price` in ascending order.

Q15. Retrieve all products sorted by `rating` in descending order, but show only the top 2.

Q16. Skip the first result and return the next 2 products sorted by `price` ascending. (Simulate page 2 of a results list.)

Q17. Create an index on the `name` field of the `products` collection.

Q18. Create a **unique** index on the `name` field. What happens if you try to insert a duplicate product name after creating it?

Before starting this section, load the employee dataset from the provided `employees.json` file:

```
mongoimport --db shopDB --collection employees --file employees.json --jsonArray
```

Q19. Count the total number of employees grouped by their `department`.

Q20. Find the **average salary** for each `department`, and sort the results from highest to lowest average.

Q21. Filter only `"active"` employees, then group by `department` to find the **total salary budget** per department.

Q22. Find the **top 3 highest-paid** employees. Return only their `name`, `role`, `department`, and `salary`.

Q23. Group employees by `department` and count how many are in each. Then sort by count descending, and only return departments with **more than 2 employees**.

Q24. For each department, find the **maximum salary** and **minimum salary**.

Q25. Explore commands lookup, unwind & watch.